

CS330 Operating Systems and Lab.

Virtualization

Spring 2026



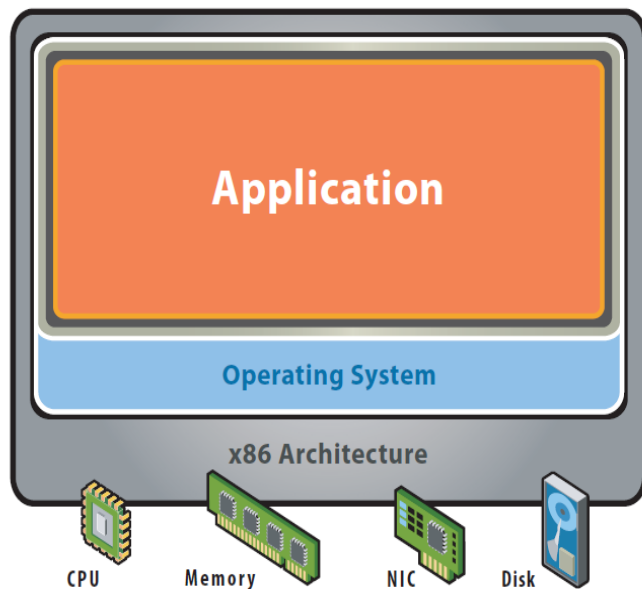
Instructor : Insik Shin

Concept of Virtualization

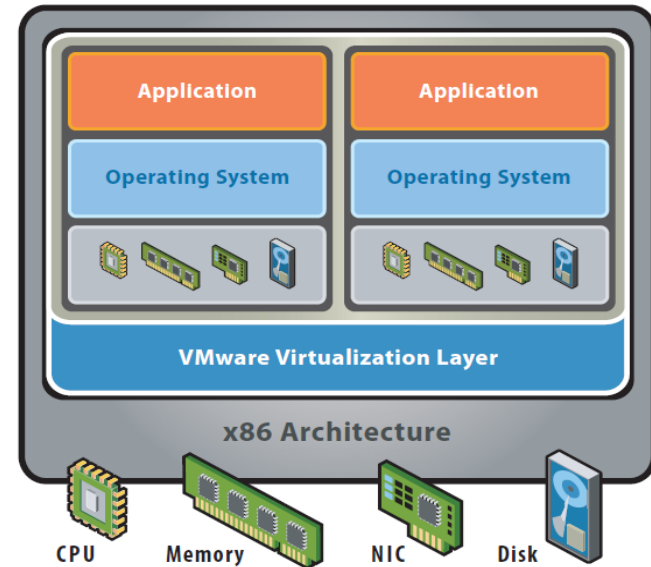
- What is virtualization?
 - The common theme of all virtualization technologies is the **hiding of technical detail**, through encapsulation,
- Typical examples of virtualization?
 - multiplexing access
 - a single physical resource appears to function as multiple logical resources
 - combining resources
 - multiple physical resources are combined to act as a single logical resource
 - simplifying control
 - simplifying the management and control of resources (even in one-to-one)

Concept of Virtualization

- “(OS) Virtualization” refers to the creation of a virtual machine
 - A virtual machine is a software implementation of a machine (i.e., a computer) that executes programs like a physical machine



Before Virtualization



After Virtualization

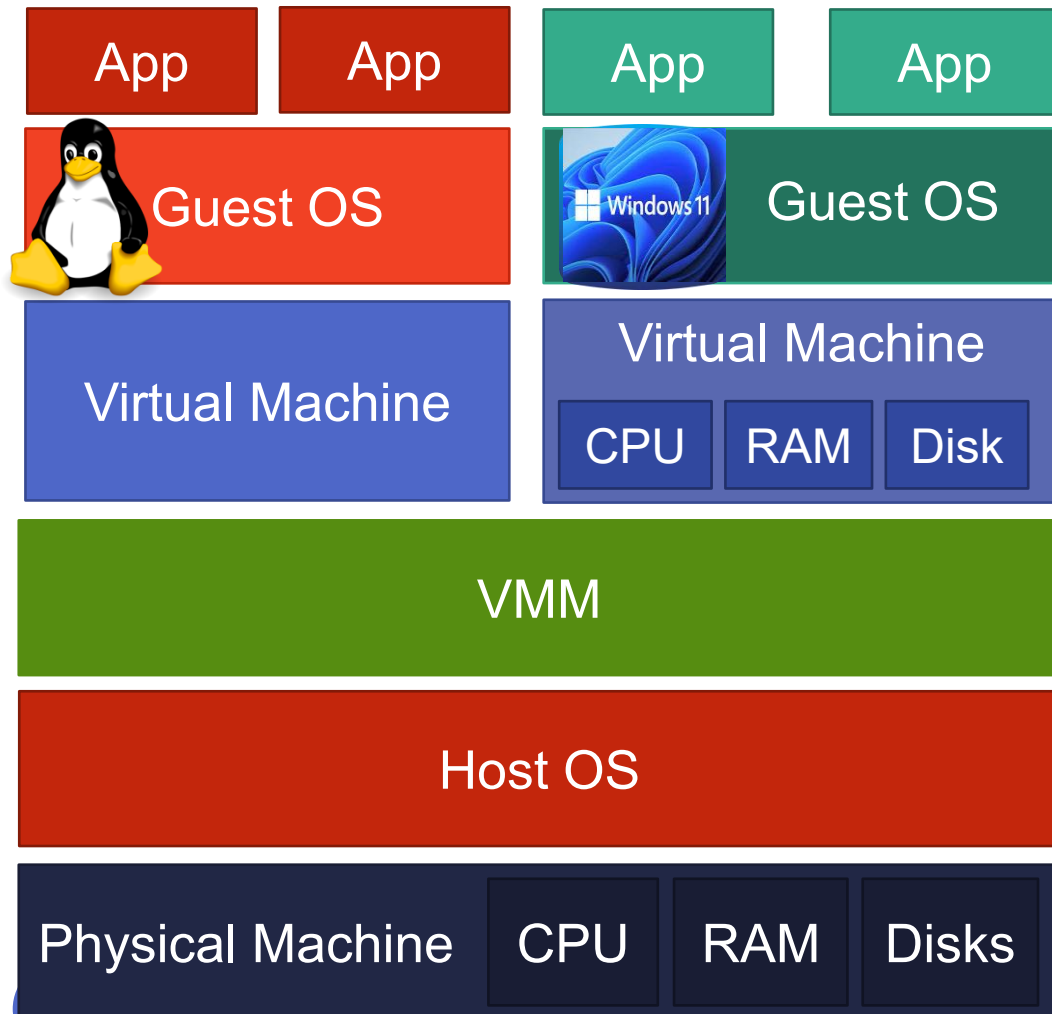
OS Fundamentals



- OS multiplexes resources between apps
- OS enforces isolation & protection between apps (ring 3)

- The OS manages physical resources
- The OS expects to have privileged access (ring 0)

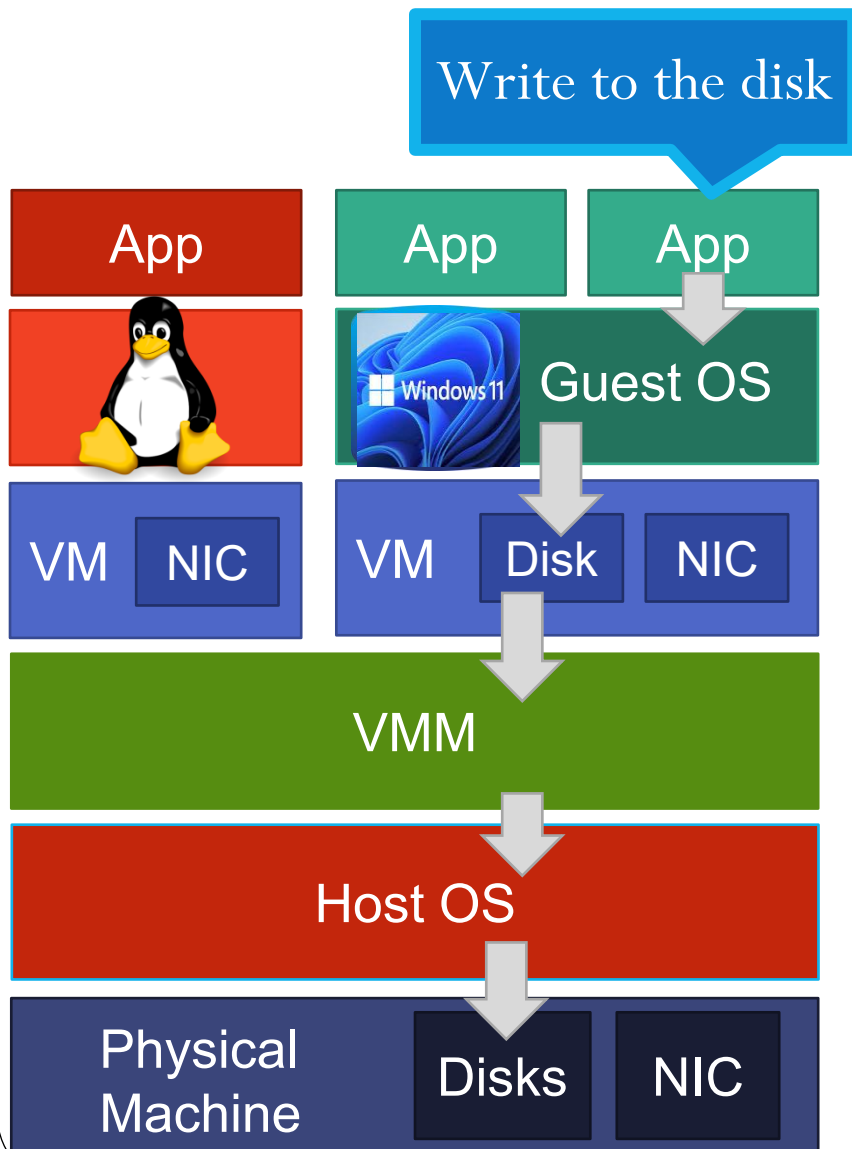
VMM Organization and Functions



- Map operations on virtual hw. to physical hw.
- Multiplex resources between guest OSes
- Enforce protection & isolation between guest OSes

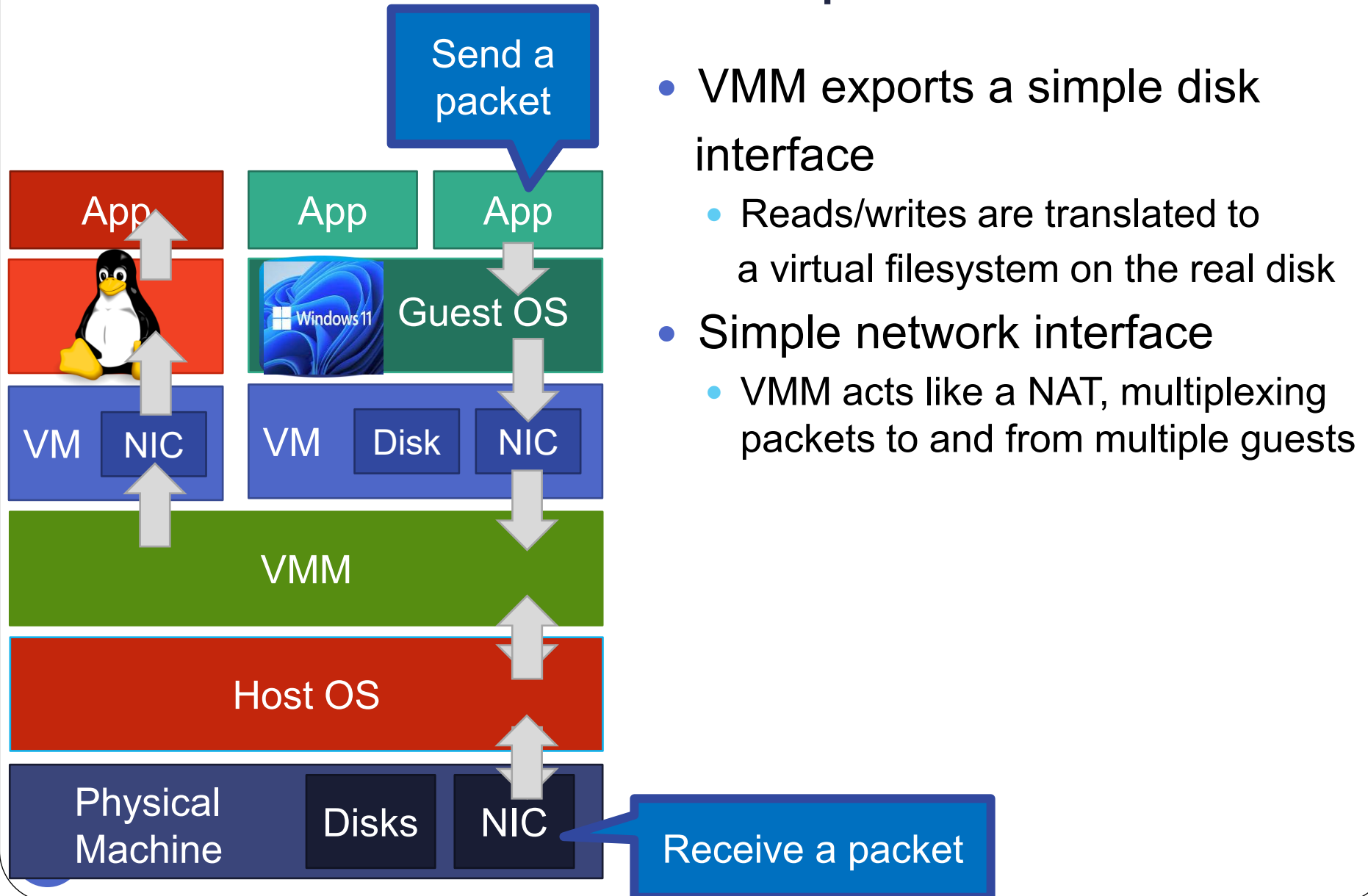
(similar to what OS does)

Virtual Hardware Examples



- VMM exports a simple disk interface
 - Reads/writes are translated to a virtual filesystem on the real disk

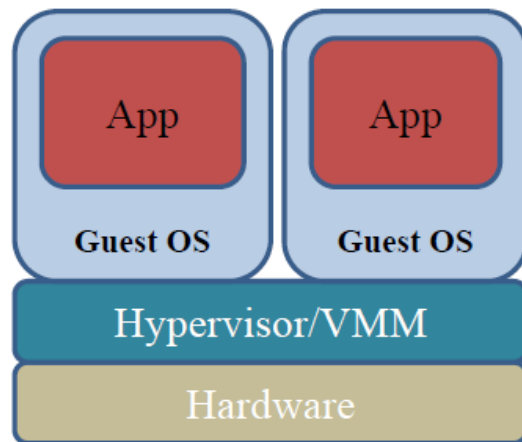
Virtual Hardware Examples



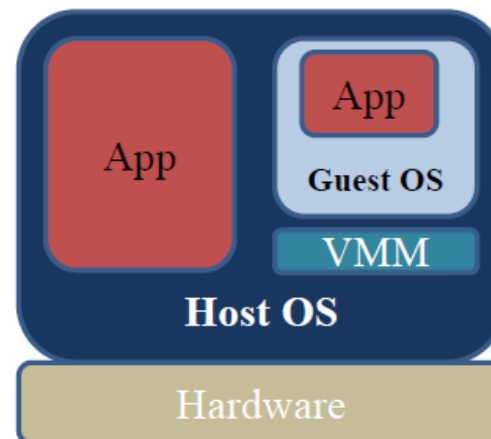
Virtualization Architecture

- Type 1 (native/bare-metal)
 - Hypervisor runs directly on the host's hardware to control the hardware and to monitor guest OSes. A guest OS runs on another level above the hardware
- Type 2 (hosted)
 - Hypervisor runs within a conventional operating system environment. (with in “Host OS”)

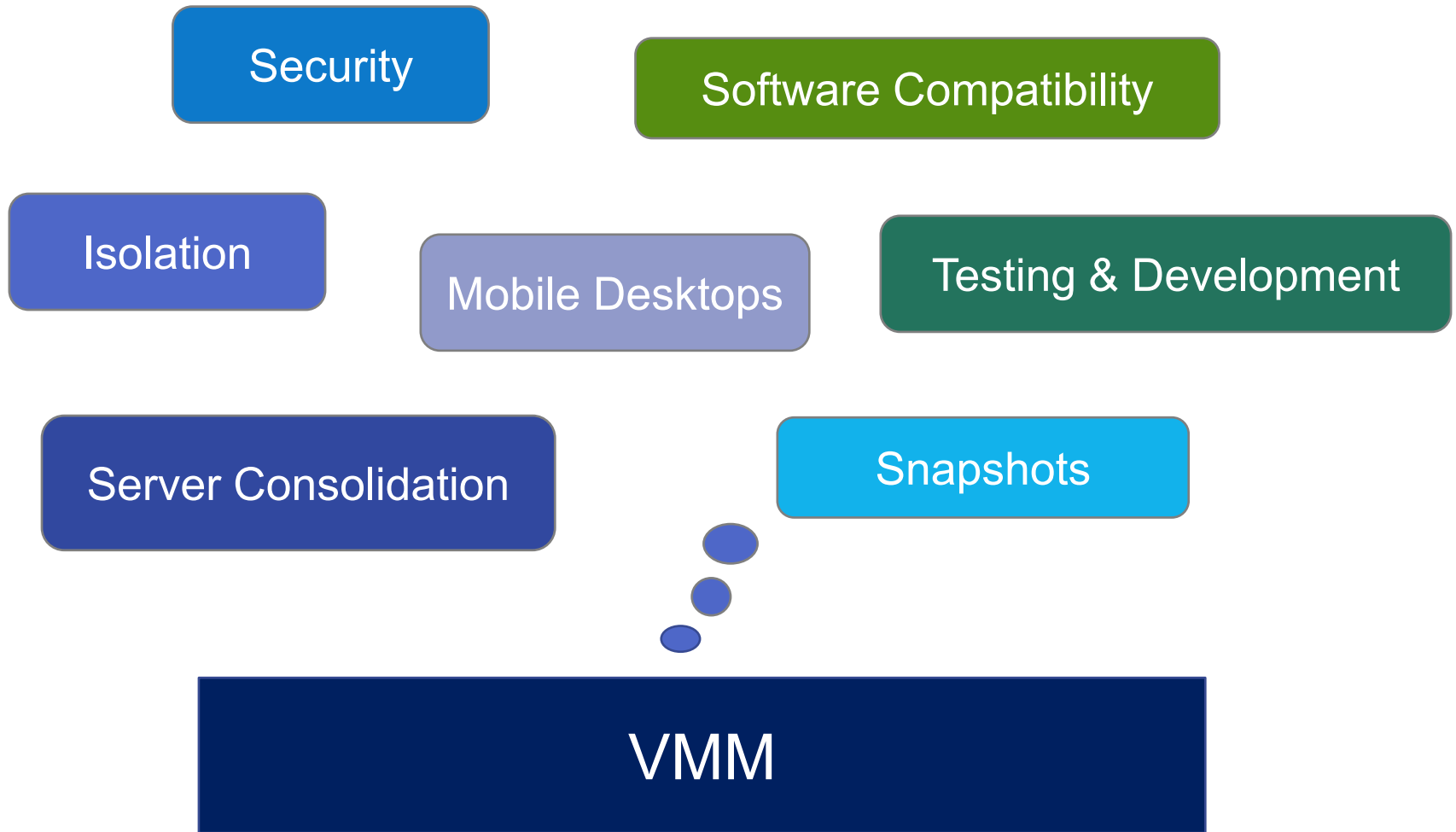
Type1(or native, bare-metal)



Type2 (or hosted)



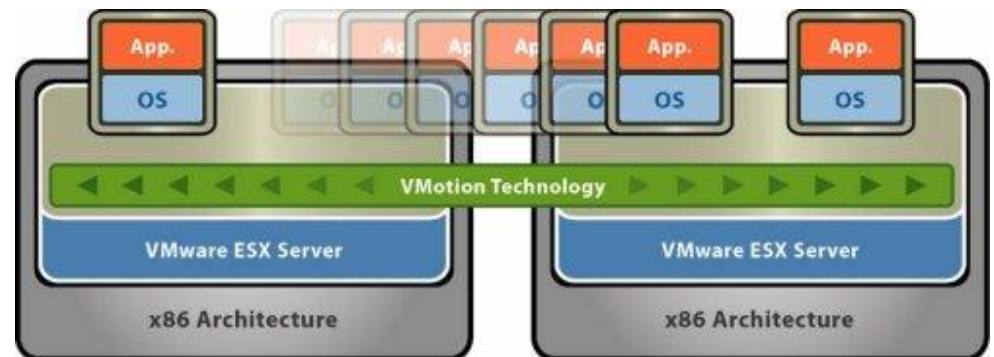
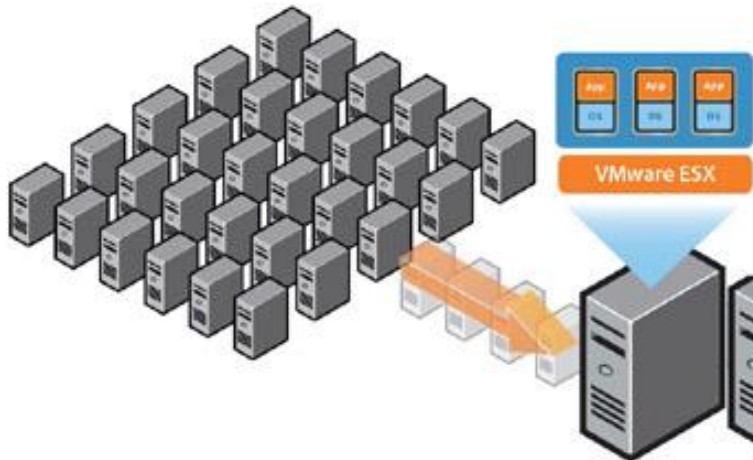
Virtulization is Everywhere



Advantages of Virtualization (1)

- Compatibility and functionality
 - Guests are oblivious to low-level hardware changes
 - Windows apps on Linux or vice-versa
- Consolidation
 - Multiple machines can be combined into one by running the OSes as guests

Server Consolidation with VMware ESX

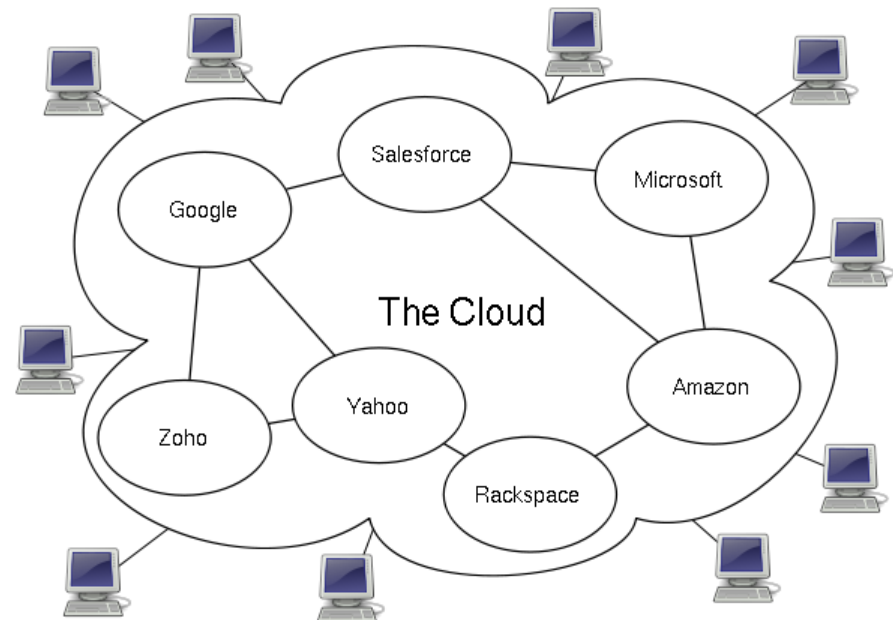


Advantages of Virtualization (2)

- Isolation, security and reliability of applications
 - A virtual machine for each (potentially untrusted) app
 - If a virtual machine crashes, all others are unaffected.
 - Data does not leak across virtual machines
- Checkpointing and migration
 - A guest OS can be written to disk or sent across the network, reloaded later or on a different machine
- OS debugging
 - Crash on developing OS do not (typically) affect Host OS
 - Debug : Memory dumps, CPU Registers, Serial / Parallel Port dumps
- Multiplatform debugging
 - App writers often target multiple platforms
 - E.g. Android, Windows, and Linux
 - Would you rather debug on three separate machines, or one machine with two guests?

Cloud Computing

- Cloud computing
 - The delivery of various computing services over the internet, allowing users to access and use computing resources without owning and maintaining physical hardware and infrastructure.
 - These services include storage, processing power, databases, networking, software, and more.
 - Similar to the electricity grid

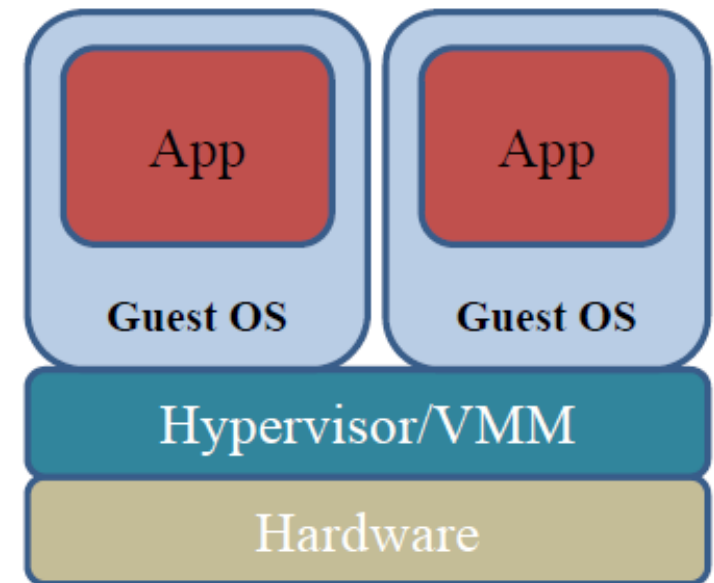


Cloud computing

- Cloud computing takes virtualization to the next step
 - You don't have to own hardware.
 - You can “rent” it as needed from a cloud.
 - Public cloud
 - e.g. Amazon EC2.
 - Also a company can create a private cloud.

Types of virtualization

- Full Virtualization
 - No Guest OS modification
 - Software approach
 - Binary translation
 - VMware
 - Hardware-assisted Virtualization
 - New privilege mode
- Paravirtualization
 - Guest OS modification
 - Software approach
 - Hypercalls
 - Xen



Outline

- Classic Virtualization
 - Trap-and-Emulate
- Software Virtualization
- Intel/AMD Hardware Virtualization

Trap-and-Emulate Virtualization

Important ideas

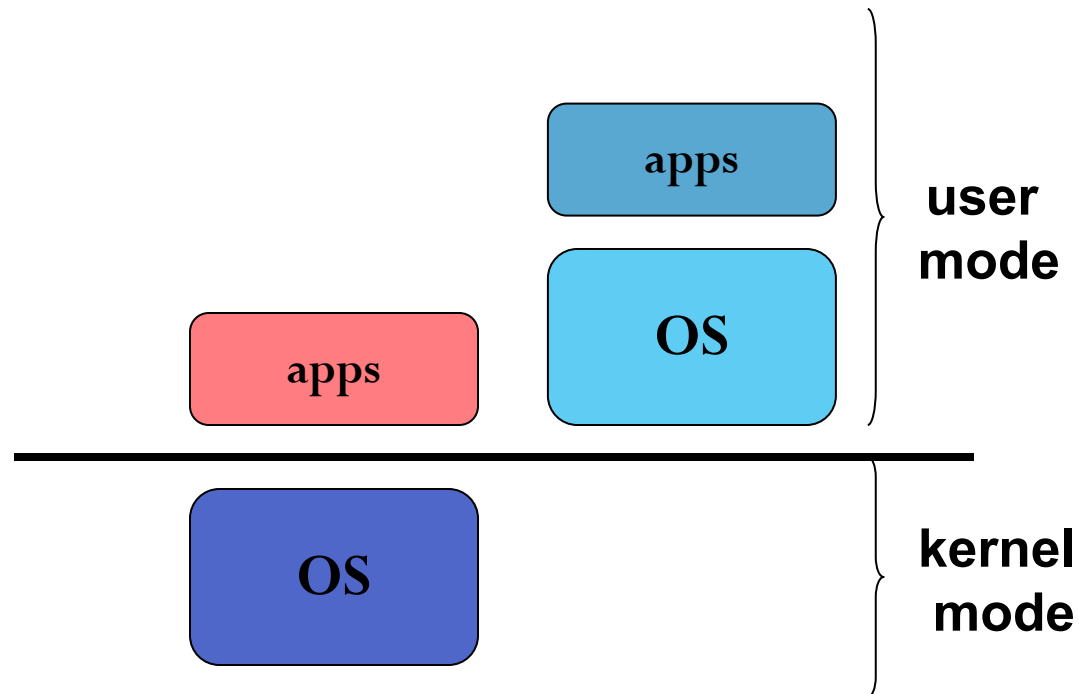
1. De-Privilege OS
2. Maintain shadow structures
3. Use of memory traces

Trap-and-Emulate Virtualization

Important ideas

1. De-Privilege OS

- Executes guest operating systems directly but at a lesser privilege level, *user-level*

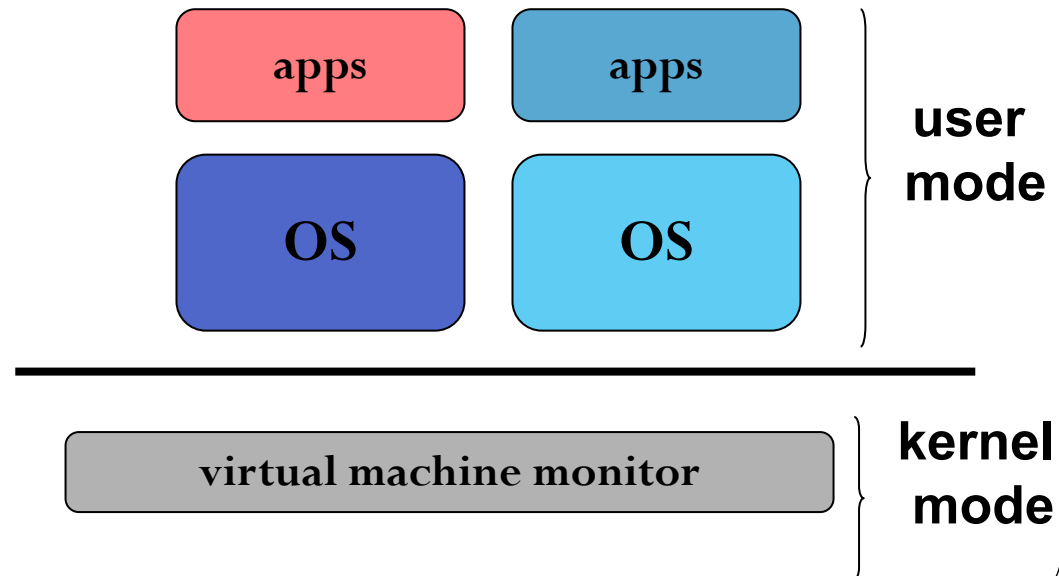


Trap-and-Emulate Virtualization

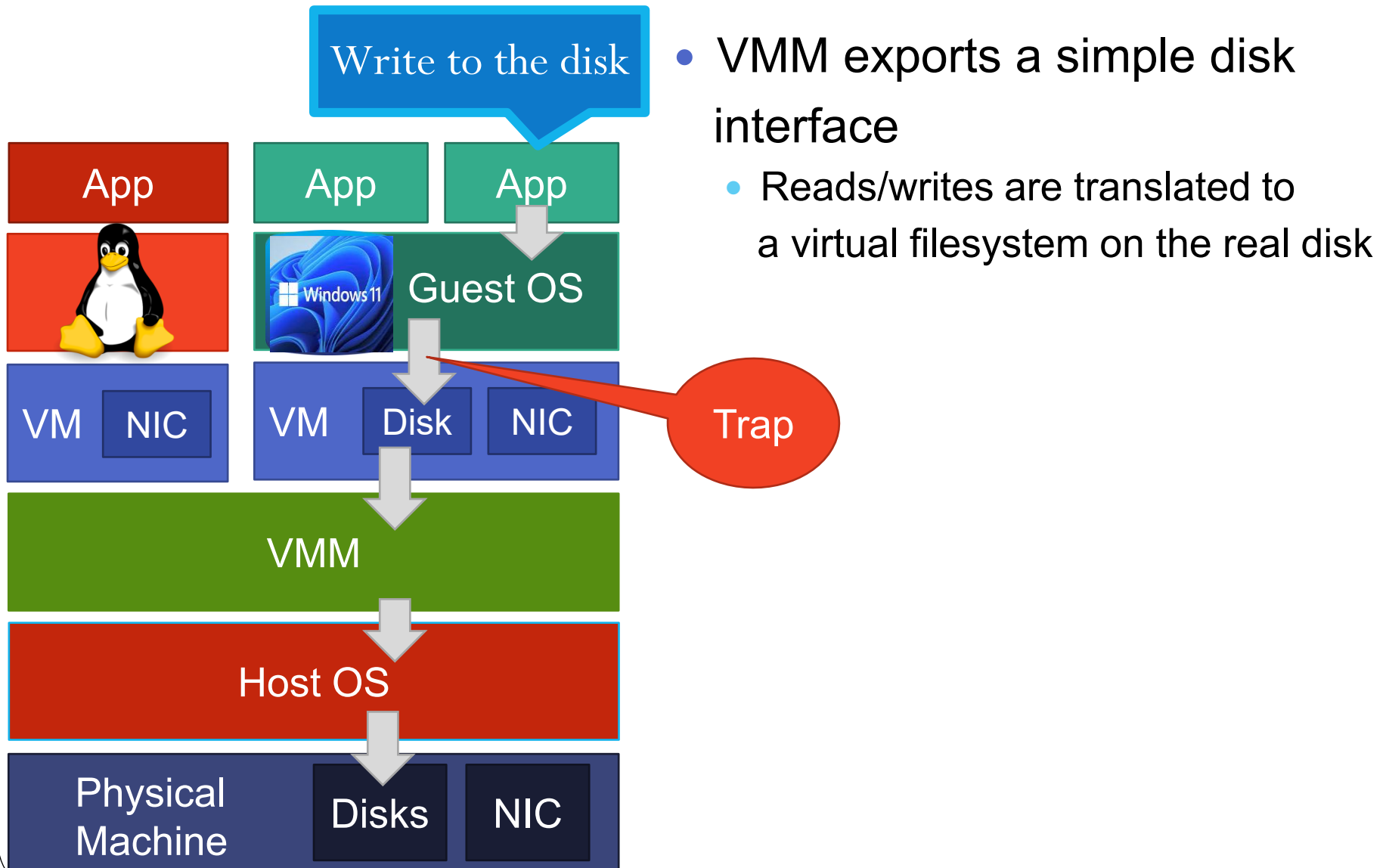
Important ideas

1. De-Privilege OS

- Executes guest operating systems directly but at a lesser privilege level, *user-level*
- Instructions that read or write privileged state – *trap*
- VMM intercepts the trap and emulates the trapping instruction against the virtual machine state



Virtual Hardware Examples

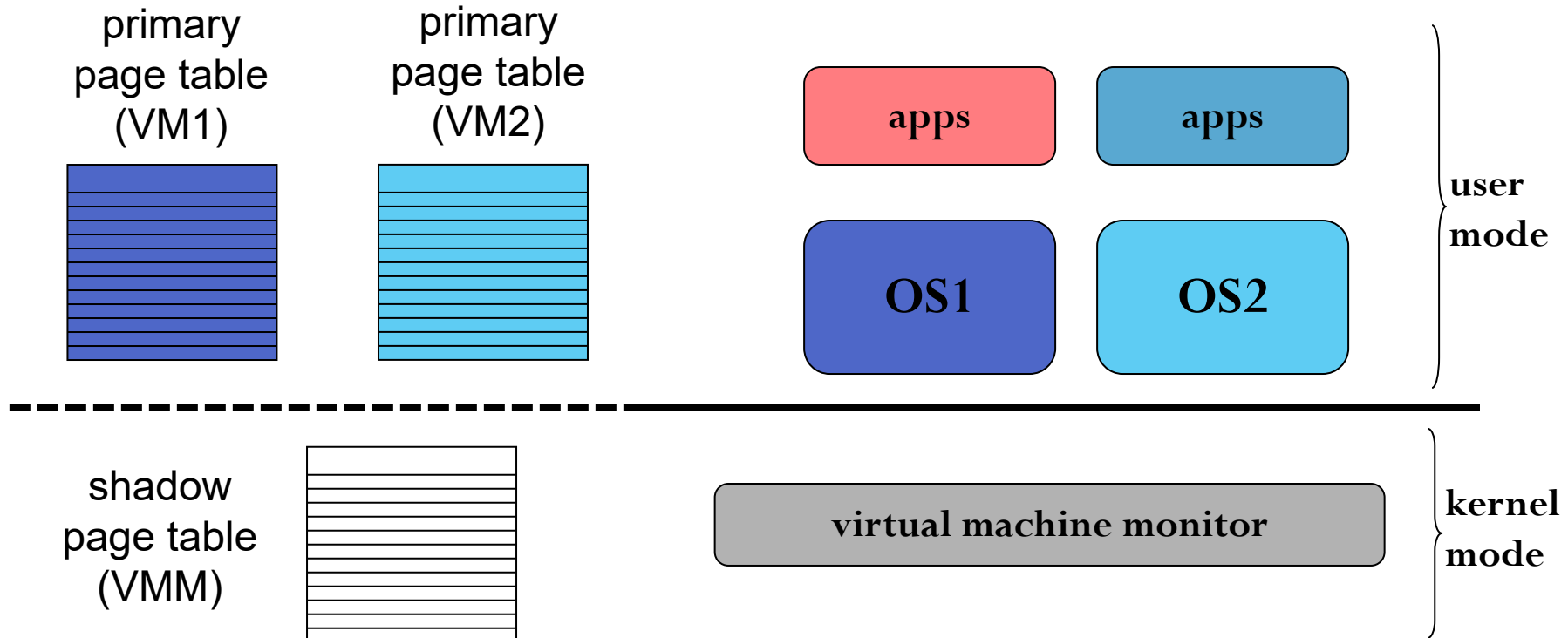


Trap-and-Emulate Virtualization

1. De-Privilege OS

2. Shadow structures and memory tracing

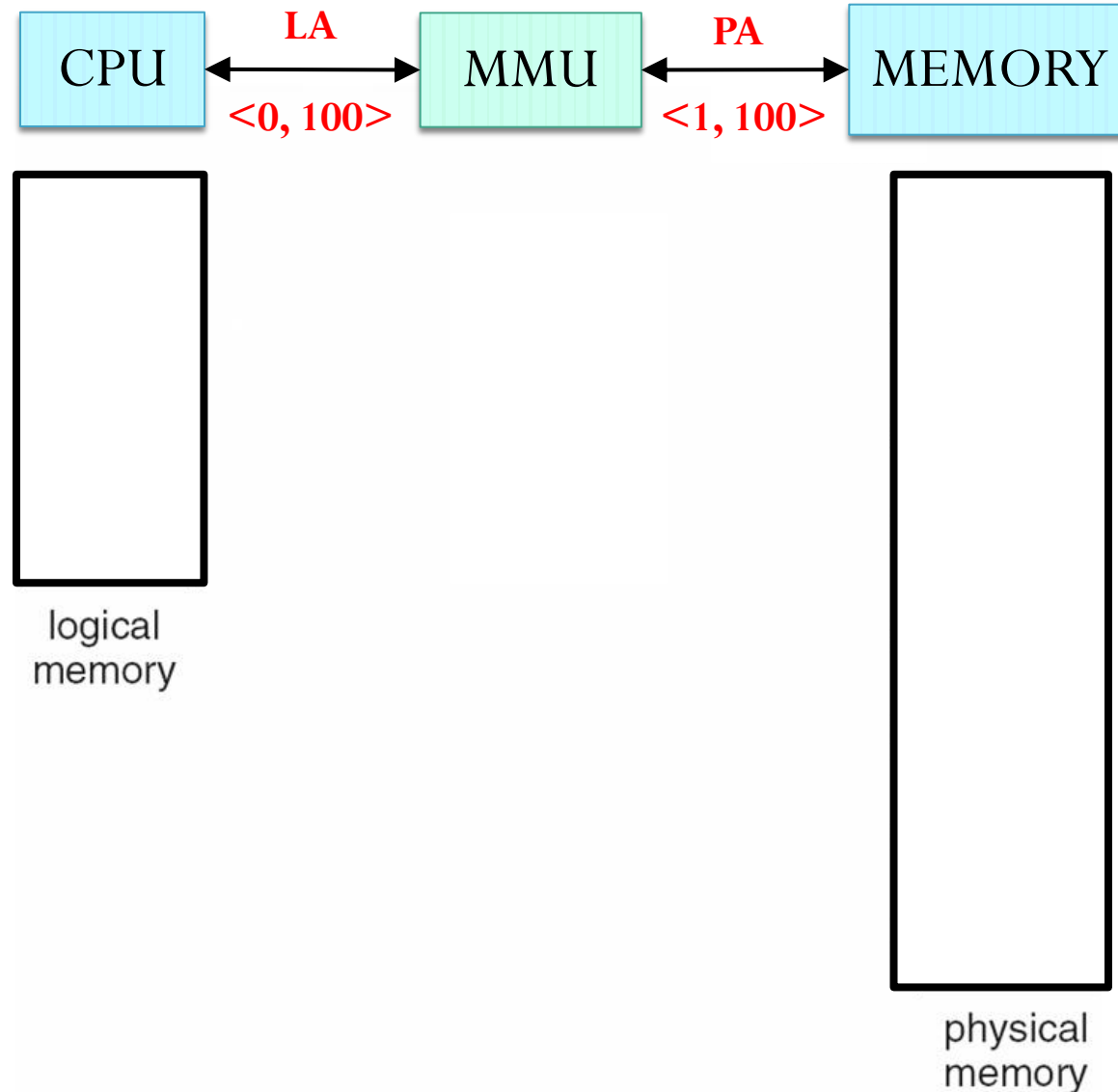
- Primary structures reflect the state of guest
- VMM-level shadow structures are copies of guest's primary structures

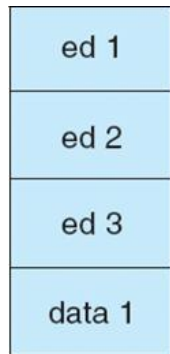


Problem: How to Virtualize the MMU?

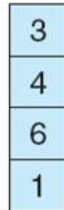
- On x86, each OS expects that it can create page tables and install them in the *cr3* register
 - The OS believes that it can access physical memory
- However, virtualized guests do not have access to physical memory
- Using trap (or binary translation), the VMM can replace writes to *cr3*
 - Store the guest's root page in the virtual CPU *cr3*
 - The VMM can now walk to guest's page tables
- However, the guest's page tables cannot be installed in the physical CPU...

Problem: How to virtualize MMU?

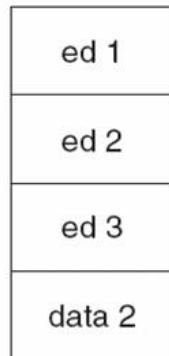




process P_1



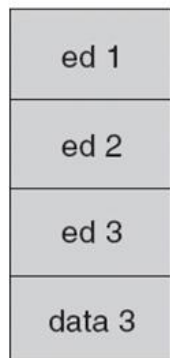
page table
for P_1



process P_2



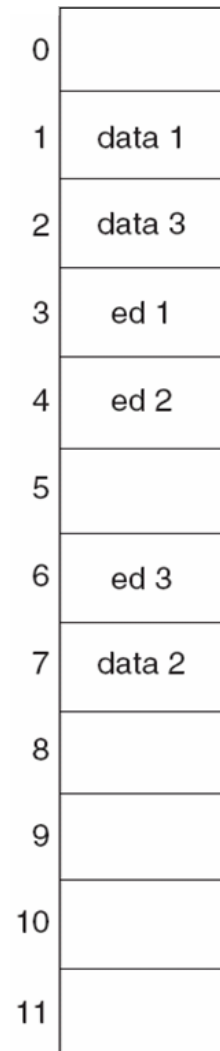
page table
for P_2



process P_3

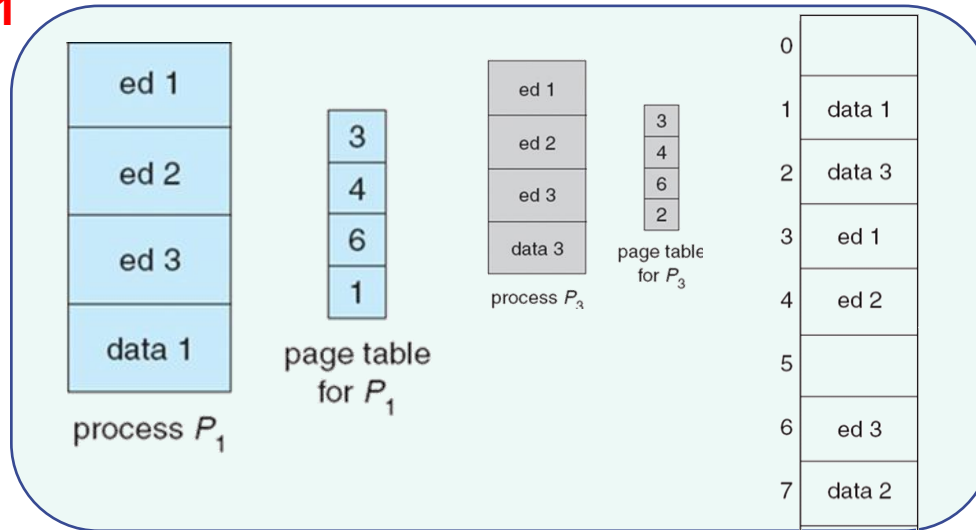


page table
for P_3

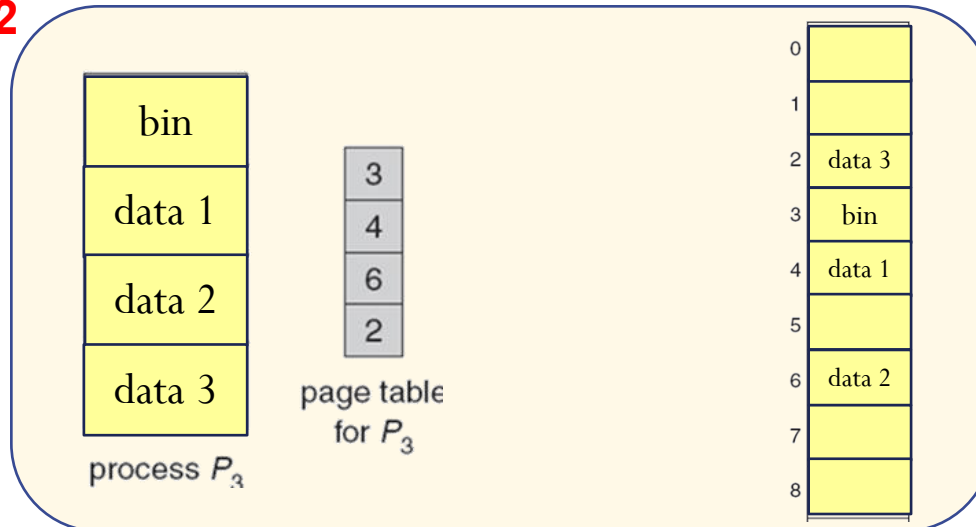




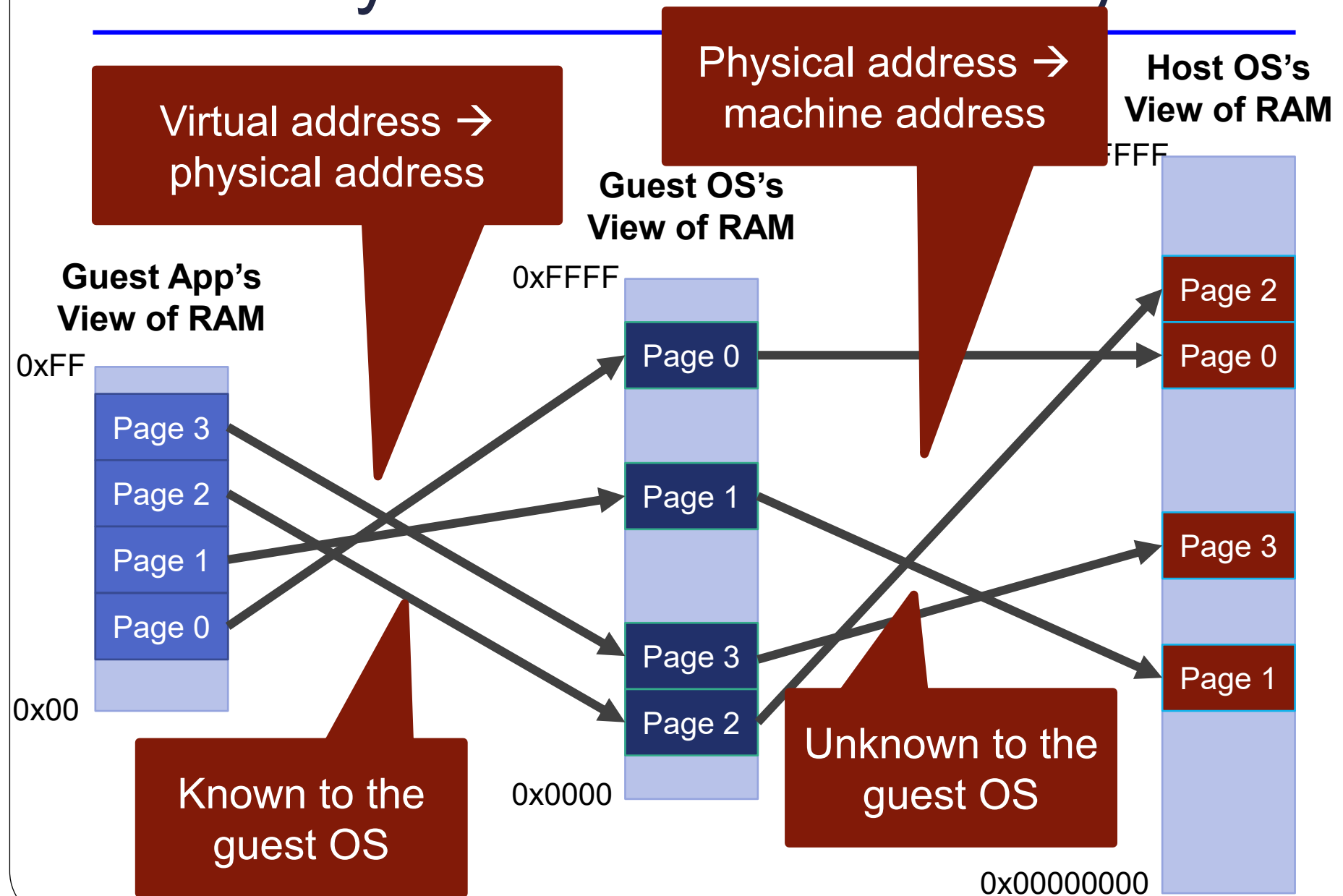
VM1



VM2



Two Layers of Virtual Memory

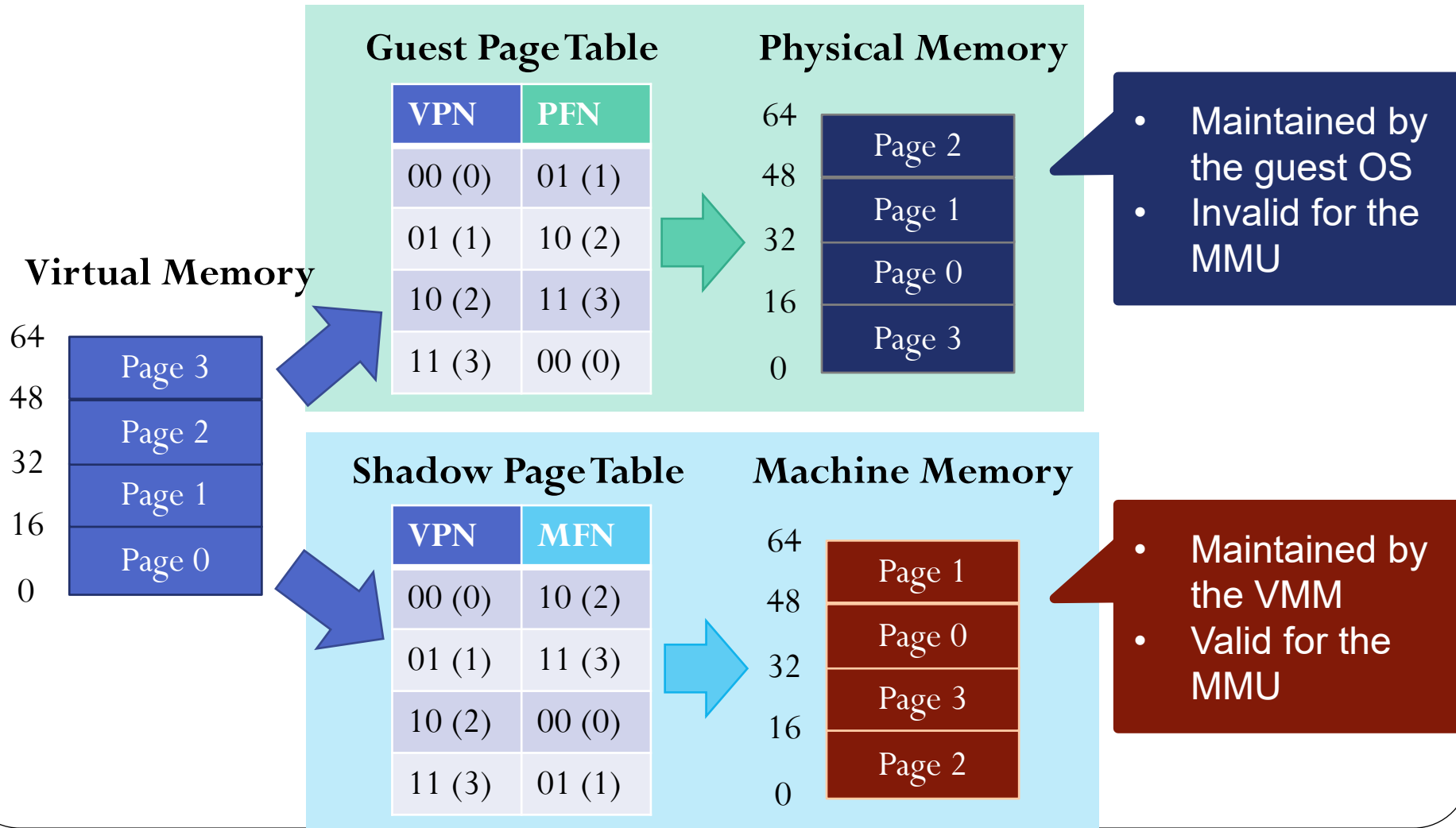


Guest's Page Tables Are Invalid

- Guest OS page tables map **virtual page numbers (VPNs)** to **physical frame numbers (PFNs)**
- Problem: the guest is virtualized, doesn't actually know the true PFNs
 - The true location is the **machine frame number (MFN)**
 - MFNs are known to the VMM and the host OS
- Guest page tables cannot be installed in *cr3*
 - Map VPNs to PFNs, but the PFNs are incorrect
- How can the MMU translate addresses used by the guest (VPNs) to MFNs?

Shadow Page Tables

- Solution: VMM creates **shadow page tables** that map VPN → MFN (as opposed to VPN→PFN)

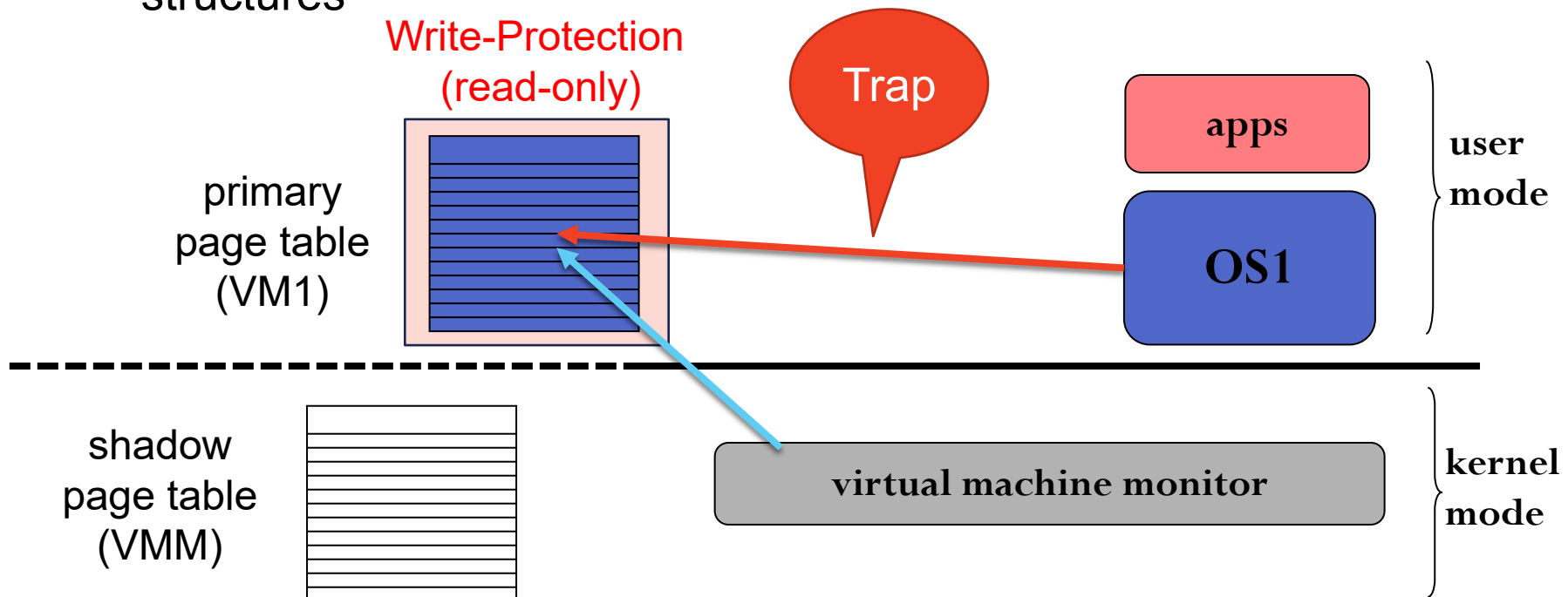


Building Shadow Tables

- Problem: how can the VMM maintain consistent shadow pages tables?
 - The guest OS may modify its page tables at any time
 - Modifying the tables is a simple memory write, not a privileged instruction
 - Thus, no helpful CPU exceptions :(
- Solution: mark the hardware pages containing the guest's tables as read-only
 - If the guest updates a table, an exception is generated
 - VMM catches the exception, examines the faulting write, updates the shadow table

Classic VMM : Memory Tracing Example

- How to do memory tracing?
 - Guest PTEs for which shadow PTEs are constructed may be writeprotected
 - Guest access to these cause traps (tracing faults)
 - VMM decodes the faulting guest instruction, emulates its effect on primary structure and propagates the change to shadow structures

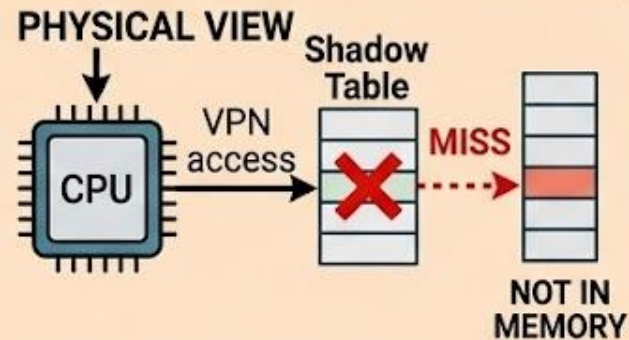
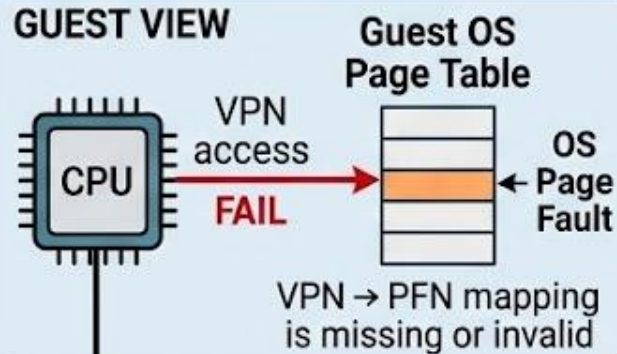


Dealing With Page Faults

- Interesting issue: who takes care of page fault? Guest or VMM?
 - It is possible that the shadow table may be inconsistent
- If a guest page faults, this could be a:
 - **True miss**: actual page fault (from the viewpoint of Guest OS as well)
 - **Hidden miss**: the shadow table is inconsistent (Guest OS not knowing the page fault)
 - there is a valid VPN→PFN mapping in the guest's page tables
- VMM must disambiguate true and hidden misses
 - On each page fault, the VMM must walk the guest's tables to see if a valid VPN→PFN mapping exists
 - If so, this is a hidden miss
 - Update the shadow table and retry the instruction
 - Otherwise, forward the page fault to the guest OS's handler

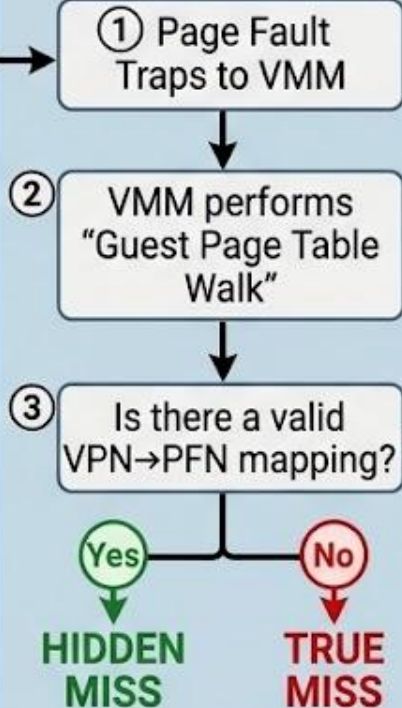
VMM Page Fault Disambiguation: True vs. Hidden Miss

Scenario 1: True Miss (Actual Page Fault)



GUEST OS SAYS:
"I don't have this page."
VMM forwards to Guest Handler.

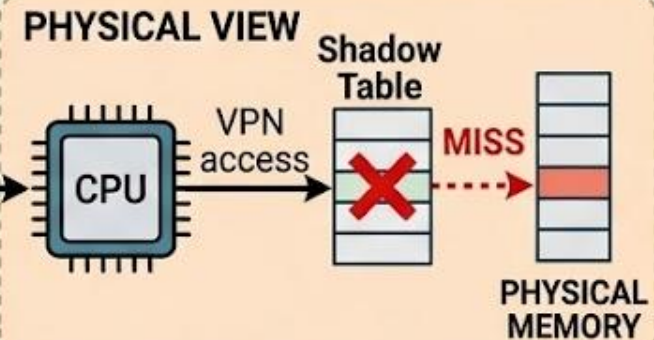
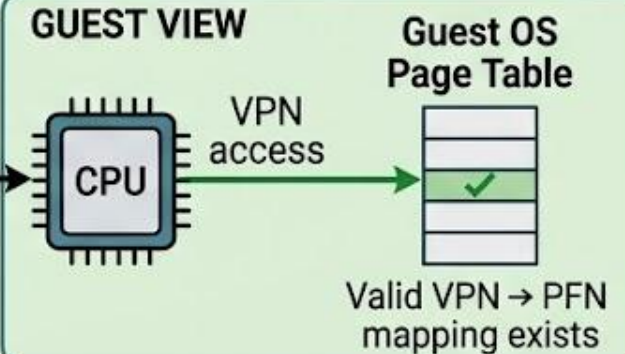
VMM Disambiguation Process



VMM updates Shadow Table
↓
Retry Instruction

VMM forwards page fault to Guest OS

Scenario 2: Hidden Miss (Inconsistent Shadow Table)



GUEST OS SAYS:
"I thought I had this."
Shadow Table inconsistent.
VMM syncs with Guest OS, updates Shadow Table, and retries instruction.

Pros and Cons

- The good: shadow tables allow the MMU to directly translate guest VPNs to hardware pages
 - Thus, guest OS code and guest apps can execute directly on the CPU
- The bad:
 - Double the amount of memory used for page tables
 - i.e. the guest's tables and the shadow tables
 - Context switch from the guest to the VMM every time a page table is created or updated
 - Very high CPU overhead for memory intensive workloads

More VMM Tricks

- The VMM can play tricks with virtual memory just like an OS can
- Paging:
 - The VMM can page parts of a guest, or even an entire guest, to disk
 - A guest can be written to disk and brought back online on a different machine!
- Shared pages:
 - The VMM can share read-only pages between guests
 - Example: two guests both running Windows XP

Trap-and-Emulate (cont.)

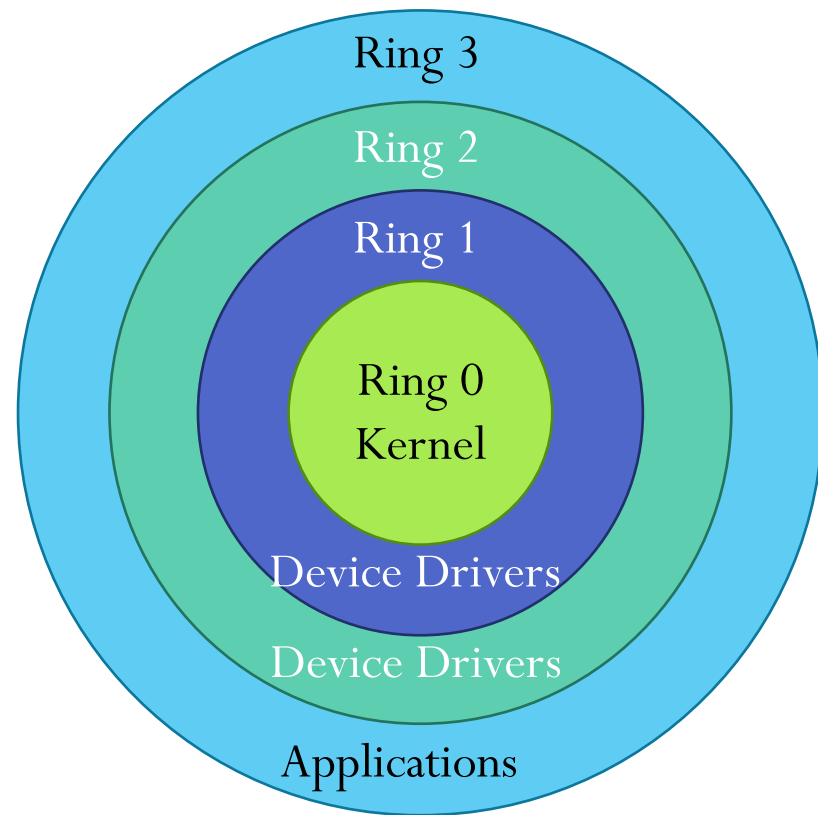
- Traps are expensive (~3000 cycles)
- Important enhancements
 - “Paravirtualization” to reduce traps (e.g., Xen)
 - Modify guest OS to provide higher-level information to VMM
 - Approach relaxes Popek and Goldberg’s fidelity requirement
 - But gains in performance Virtual Machines 10
 - Hardware VM modes (e.g., IBM s370)

Outline

- Classic Virtualization
- Software Virtualization
- Intel/AMD Hardware Virtualization

Protected Mode of x86 Machines

- Most modern CPUs support **protected mode**
- x86 CPUs support three rings with different privileges
 - Ring 0: OS kernel
 - Ring 1, 2: device drivers
 - Ring 3: userland
- Most OSes only use rings 0 and 3



Can x86 Trap and Emulate?

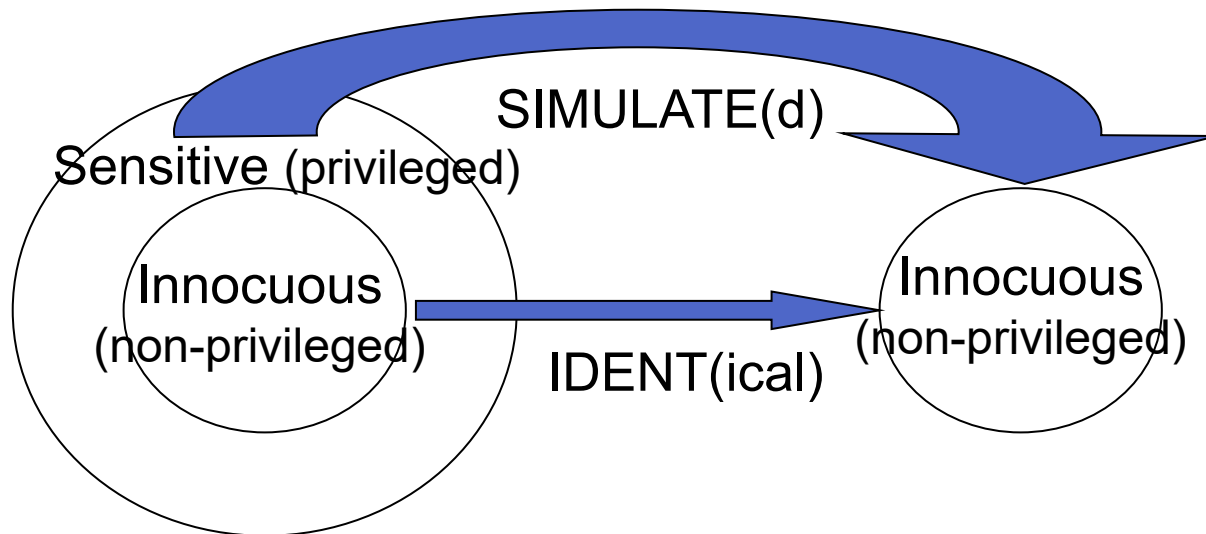
- No
 - Even with 4 execution modes!
 - Key problem: *dual-purpose instructions* don't trap
- x86 obstacles to classical virtualization
 - Lack of traps when privilege instructions run at user-level
 - Classic Example: ***popf*** instruction
 - Same instruction behaves differently depending on execution mode
 - User Mode: changes ALU flags
 - Kernel Mode: changes ALU **and** system flags
 - **Does not generate a trap in user mode**

Software VMM: To overcome obstacles on x86

- Execute guest on Interpreter
 - But fetch-decode-execute cycle of interpreter reduces performance significantly

Software VMM: To overcome obstacles on x86

- Execute guest on Interpreter
 - But fetch-decode-execute cycle of interpreter reduces performance significantly
- Binary Translation of the guest
 - VMM can switch between BT mode and direct execution mode
 - Privileged instructions are changed to function calls to code in VMM



Innocuous : not harmful

Binary Translation Example

- Guest OS Assembly

do_atomic_operation:

cli

mov eax, 1

xchg eax, [lock_addr]

test eax, eax

jnz spinlock

...

...

mov [lock_addr], 0

sti

ret

- Translated Assembly

do_atomic_operation:

call [vmm_disable_interrupts]

mov eax, 1

xchg eax, [lock_addr]

test eax, eax

jnz spinlock

...

...

mov [lock_addr], 0

call [vmm_enable_interrupts]

ret

Binary Translation (VMWare)

- Characteristics
 - Binary – input is machine-level code
 - Dynamic – occurs at runtime
 - On demand – code translated when needed for execution
 - System level – makes no assumption about guest code
 - Only need to translate OS code
 - Makes applications run fast by default
 - Most instruction sequences don't change
 - Instructions that **do change**:
 - Indirect control flow: call/ret, jmp
 - PC-relative addressing
 - Privileged instructions

Pros and Cons

- Advantages of binary translation
 - It makes it safe to virtualize x86 assembly code
 - Translation occurs dynamically, on demand
 - No need to translate the entire guest OS
 - App code running in the guest does not need to be translated
- Disadvantages
 - Translation is slow
 - Wastes memory (duplicate copies of code in memory)
 - Translation may cause code to be expanded or shortened
 - Thus, **jmp** and **call** addresses may also need to be patched

Outline

- Classic Virtualization
- Software Virtualization
- Intel/AMD Hardware Virtualization

The Story So Far...



- We have discussed how systems like VMWare implement full virtualization
- Key challenges solved by VMWare:
 - Binary translation rewrites guest OS assembly to not use privileged instructions
 - Shadow page tables maintained by the VMM allow the MMU to translate addresses for guest code
- So what's the problem?
 - **Performance**

Virtualization Performance

- Guest code executes on the physical CPU
- However, that doesn't mean it's as fast as the host OS or native applications
 1. Guest code must be binary translated
 2. Shadow page tables must be maintained
 - Page table updates cause expensive context switches from guest to VMM
 - Page faults are at least twice as costly to handle

Hardware Techniques

- Modern x86 chips support hardware extensions designed to improve virtualization performance
 1. Reliable exceptions during privileged instructions
 - Known as AMD-V and VT-x (Intel)
 - Released in 2006
 - Adds **vmenter/vmexit** instructions (like **sysenter/sysret**)
 - allows classic trap-and-emulate
 2. Extended page tables for guests
 - Known as RVI (AMD) and EPT (Intel)
 - Adds another layer onto existing page table to map PFN→MFN

AMD-V and VT-x

- Annoyingly, AMD and Intel offer different implementations
- However, both offer similar functionality
- **vmexit**: exception thrown by the CPU if the guest executes a privileged instruction
 - Saves the running state of the guest's CPU
 - Context switches back to the VMM
- **vmenter**: instruction used by the hypervisor to context switch into a guest
 - Downgrade CPU privilege to ring 3

Benefits of AMD-V and VT-x

- Greatly simplifies VMM implementation
 - No need for binary translation
 - Simplifies implementation of shadow page tables
- Warning: the VMM runs in userland, but use of AMD-V and VT-x requires ring 0 access
 - Host OS must offer APIs that allow VMMs to configure VMCB and setup callbacks for guest OS exceptions
 - Example: KVM on Linux

Problem with AMD-V and VT-x

- Some operations are **much** slower when using **vmexit** vs. binary translation

Guest OS Assembly

do_atomic_operation:

cli

mov eax, 1

- This code is okay because cli is trapped by vmexit
- However, each vmexit causes an expensive context switch

Translated Assembly

do_atomic_operation:

call [vmm_disable_interrupts]

mov eax, 1

...

- The VMM must generate this code via binary translation
- But, this direct call is very fast, no context switch needed

Benefits of AMD-V and VT-x

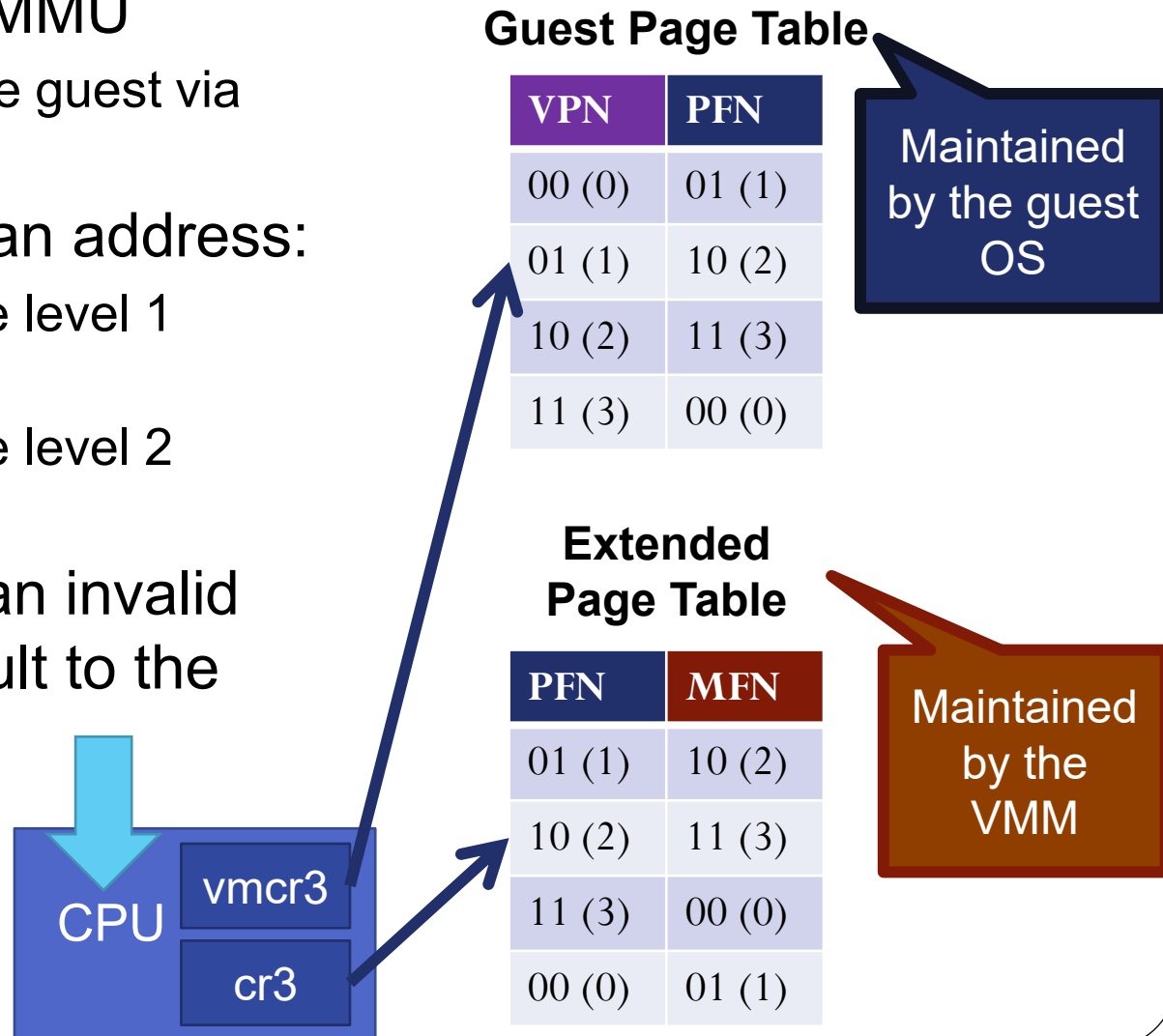
- Greatly simplifies VMM implementation
 - No need for binary translation
 - Simplifies implementation of shadow page tables
- ... however, sophisticated VMMs still use binary translation in addition to **vmenter/vmexit**
 - VMM observes guest code that causes frequent **vmexits**
 - Hot spots may be binary translated or dynamically patched to improve performance
 - Similar to Just-In-Time (JIT) compilation

Second Level Address Translation

- AMD-V and VT-x help the VMM control guests
- ... but, they don't address the need for shadow page tables
- **Second level address translation (SLAT)** allows the MMU to directly support guest page tables
 - Intel: **Extended Page Tables (EPT)**
 - AMD: Rapid Virtualization Indexing (RVI)
 - Also known as Two Dimensional Paging (TDP)
 - Introduced in 2008

SLAT Implementation

- VMM installs first and second level tables in the MMU
 - Context switch to the guest via **vmenter**
- Steps to translate an address:
 1. MMU queries the level 1 (guest) table
 2. MMU queries the level 2 (VMM) table
- If any step yields an invalid PTE then page fault to the VMM (**vmexit**)



Advantages of SLAT

- **Huge** performance advantages vs. shadow page tables
- When guests `mov cr3`, the CPU updates `vmcr3` register
 - No need to `vmexit` when guest OS switches context
- EPT can be filled on-demand or pre-initialized with PFN→MFN entries
 - On-demand:
 - Slower, since many address translations will trigger hidden misses
 - ... but hardware pages for the guest can be allocated when needed
 - And, the EPT table will be smaller
 - Preallocation:
 - No need to `vmexit` when the guest OS creates or modifies its page tables
 - ... but hardware pages need to be reserved for the guest
 - And, the EPT table will be larger

Disadvantages of SLAT

- Memory overhead for EPT
 - ... but not as much as shadow page tables
- TLB misses are twice as costly
 - SLAT makes page tables twice as deep, hence it takes twice as long to resolve PTEs